

ASSESSX

Technical Requirements Document (TRD)

Version: 1.0

Status: Initial Technical Architecture

Product: AssessX

Architecture: Modular → Scalable Distributed Architecture

Primary Client: Windows Desktop

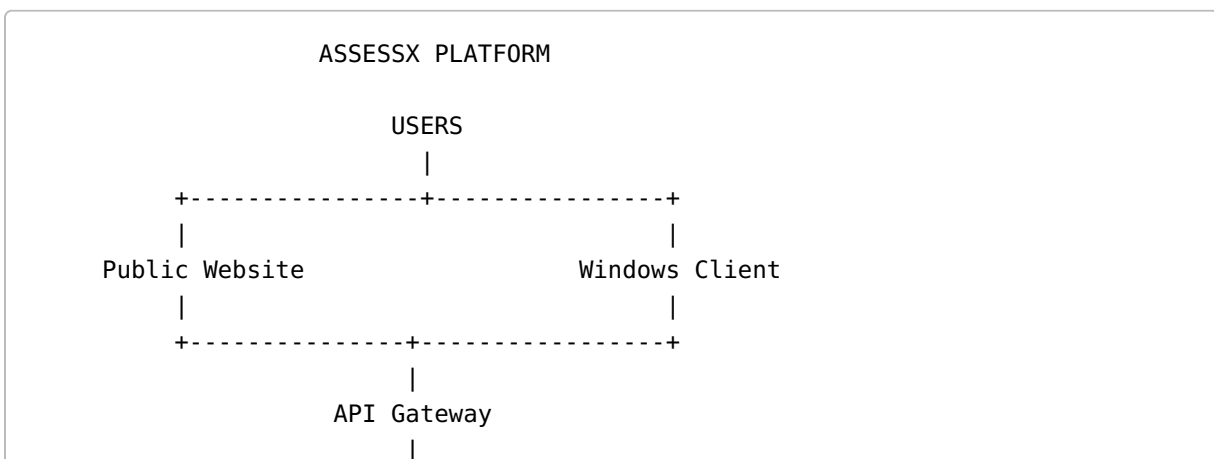
1. TECHNICAL OBJECTIVE

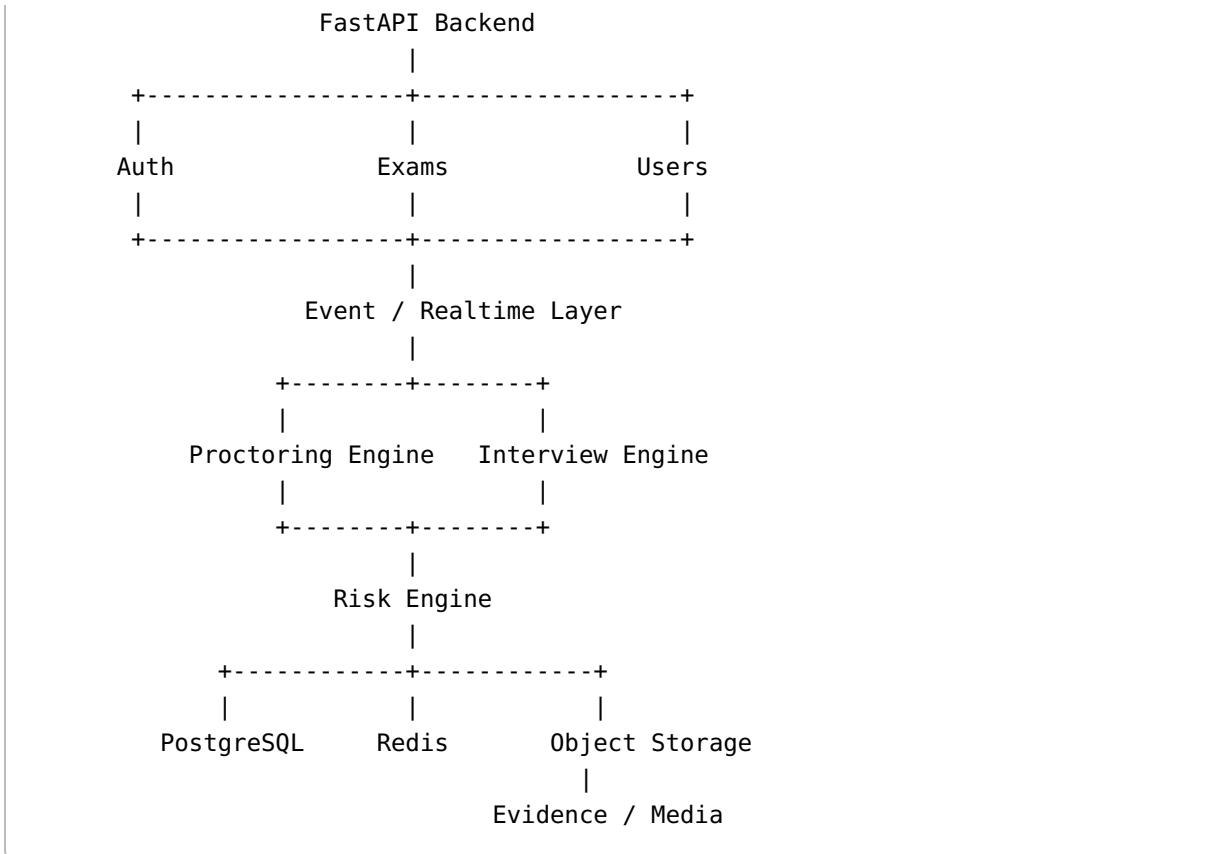
Build a secure, modular and scalable assessment platform capable of supporting:

- Online examinations
- Desktop secure-exam environments
- Multimodal AI proctoring
- Real-time event processing
- Risk scoring
- Evidence collection
- Coding assessments
- Live WebRTC interviews
- AI interviews
- Enterprise administration

The architecture must allow individual components to scale independently.

2. HIGH-LEVEL ARCHITECTURE





3. CLIENT ARCHITECTURE

3.1 Public Web

Technology:

- React
- TypeScript
- Vite
- Tailwind CSS

Responsibilities:

- Landing page
- Product information
- Download
- Documentation
- System requirements

3.2 Windows Application

Preferred:

- Tauri
- React
- TypeScript
- Rust

Native capabilities may be implemented using Rust/Windows APIs where required.

Responsibilities:

- Authentication
 - System verification
 - Secure exam mode
 - Camera
 - Microphone
 - Screen capture
 - Exam UI
 - Local security checks
 - Session communication
-

4. BACKEND

Primary backend:

Python

Framework:

FastAPI

Responsibilities:

- Authentication
- User management
- Organization management
- Exam management
- Question management
- Candidate management
- Exam sessions
- Results
- Proctoring events

- Risk scores
- Interviews
- Reports

Use:

- Pydantic
 - SQLAlchemy
 - Alembic
-

5. DATABASE

Database:

PostgreSQL

Primary entities:

```
User
Organization
Role
Exam
Question
QuestionOption
ExamQuestion
CandidateExam
ExamAttempt
Answer
Submission
ProctoringSession
ProctoringEvent
Evidence
RiskScore
Interview
InterviewParticipant
InterviewEvent
InterviewEvaluation
Report
AuditLog
```

Use UUIDs for externally exposed identifiers where appropriate.

Use database constraints and indexes.

6. DATABASE DESIGN PRINCIPLES

Every tenant-sensitive table must be associated with an organization where applicable.

Never rely solely on frontend filtering for tenant isolation.

Authorization must be enforced server-side.

Use:

- Foreign keys
- Unique constraints
- Check constraints
- Indexes
- Transactions

Avoid excessive database queries.

7. REDIS

Redis is used for:

- Caching
- Short-lived session state
- Rate limiting
- Pub/sub
- Realtime state
- Event queues where appropriate

Redis must NOT become the permanent source of truth for critical exam results.

PostgreSQL remains authoritative.

8. EVENT ARCHITECTURE

Proctoring should be event-driven.

Example:

Camera
|

```
CV Processing
|
Detection
|
Structured Event
|
Redis/Event Bus
|
Risk Engine
|
Alert
|
PostgreSQL
```

Do not send every video frame to the backend.

9. AI PROCTORING PIPELINE

```
Camera
|
Frame Capture
|
Preprocessing
|
+-----+
|         |         |
| YOLO     | MediaPipe |
|         |         |
| Object Detection | Face/Pose |
|         |         |
+-----+-----+
|
| Event Builder
|
| Risk Engine
|
| Evidence Engine
|
| Alert/Storage
```

10. COMPUTER VISION

OpenCV

Responsibilities:

- Frame processing
- Image manipulation
- Camera utilities
- Preprocessing

YOLO

Responsibilities:

- Object detection

Potential classes:

- Person
- Cellphone
- Laptop
- Tablet
- Monitor
- Book
- Headphones
- Calculator

Models must be benchmarked before production.

MediaPipe

Potential:

- Face landmarks
- Pose
- Head orientation
- Gaze-related features

11. AUDIO PIPELINE

Microphone

|

```
Audio Capture
|
Voice Activity Detection
|
Speaker Analysis
|
Audio Event
|
Risk Engine
```

Potential events:

- VOICE_DETECTED
- MULTIPLE_SPEAKERS
- UNKNOWN_SPEAKER
- SUSPICIOUS_AUDIO

Noise suppression and environment calibration should be considered.

12. SCREEN MONITORING

Screen monitoring should be handled primarily by the desktop client.

Client generates structured events such as:

```
WINDOW_CHANGED
DISPLAY_CHANGED
APPLICATION_CHANGED
COPY_ATTEMPT
PASTE_ATTEMPT
SCREEN_CAPTURE_STOPPED
```

The server receives events and evaluates them.

Do not upload the complete desktop continuously unless explicitly required by the assessment policy.

13. SECURE EXAM MODE

The desktop client should enforce the strongest practical restrictions supported by the operating system.

Potential capabilities:

- Full-screen exam environment
- Restricted navigation
- Application switch detection
- Display detection
- Clipboard restrictions
- Session integrity
- Tamper detection
- Permission enforcement

Security functionality must be tested against realistic bypass attempts.

Never advertise an absolute "unbreakable" exam environment.

14. IDENTITY PIPELINE

```
Candidate
|
ID Capture
|
Face Detection
|
Face Embedding
|
Comparison
|
Liveness
|
Identity Result
```

Identity result:

```
VERIFIED
MISMATCH
FAILED
REVIEW_REQUIRED
```

Biometric data must be handled according to applicable privacy requirements.

15. RISK ENGINE

The risk engine is independent of individual AI models.

Example input:

```
{
  "event": "PHONE_DETECTED",
  "confidence": 0.94,
  "severity": "medium"
}
```

Risk engine considers:

- Event severity
- Confidence
- Frequency
- Duration
- Recency
- Correlation
- Candidate context
- Exam policy

Output:

```
{
  "risk_score": 78,
  "risk_level": "HIGH",
  "reasons": [
    "Phone detected",
    "Repeated gaze deviation",
    "Unknown voice detected"
  ]
}
```

16. RISK ENGINE DESIGN

Initial version:

Rule-based.

Future:

ML-based anomaly scoring.

Example:

```
Phone = +30
Extra Person = +40
Face Missing = +15
Application Switch = +20
Unknown Voice = +20
Gaze Deviation = +5
```

These values are configuration, not permanent truths.

Repeated events may increase score.

Normal behavior may decay risk over time where appropriate.

Risk calculations must remain explainable.

17. EVIDENCE ENGINE

For suspicious events:

```
Event
|
Evidence Capture
|
Object Storage
|
Evidence Metadata
|
PostgreSQL
```

Database stores:

- Evidence ID
- Event ID
- Storage location
- Timestamp
- Type
- Hash
- Metadata

Large files should not be stored directly inside PostgreSQL.

18. OBJECT STORAGE

Preferred:

AWS S3

Store:

- Screenshots
- Video clips
- Audio evidence
- Interview recordings
- Generated reports

Use signed URLs for controlled access.

Do not expose raw storage buckets publicly.

19. REAL-TIME ALERTS

Potential architecture:

```
Proctoring Worker
|
Redis/Event Bus
|
Risk Engine
|
WebSocket
|
Admin Dashboard
```

Admin should receive events without refreshing the page.

20. WEBSOCKETS

Use WebSockets for:

- Live proctoring alerts
- Candidate status
- Risk score updates
- Interview signaling where appropriate

Do not use WebSockets as the media transport for video.

21. WEBRTC

WebRTC handles:

- Live video
- Live audio
- Screen sharing

For live interviews:

```
Candidate
|
WebRTC
|
Interviewer
```

Signaling can use:

- WebSocket
- HTTP

STUN/TURN infrastructure is required for reliable connectivity.

Possible platforms:

- Self-hosted WebRTC infrastructure
- LiveKit
- Jitsi

Technology should be selected based on scale, control and deployment requirements.

IMPORTANT:

WebRTC does not weaken the security model by itself.

It is only the realtime communication layer.

Proctoring runs independently.

22. LIVE INTERVIEW SECURITY

During an interview:

```
WebRTC Media
  +
Screen Monitoring
  +
Camera Monitoring
  +
Audio Monitoring
  +
Identity
  |
Proctoring Engine
  |
Risk Engine
```

Interviewer receives alerts.

Example:

Medium Risk — Additional person detected

The interviewer should NOT receive raw sensitive information unless permitted by policy.

23. AI INTERVIEW ARCHITECTURE

```
Question
  |
Candidate Answer
  |
Speech-to-Text
  |
LLM
```

```

|
Answer Evaluation
|
Follow-up Generation
|
Next Question

```

LLM should receive structured context rather than raw video.

24. AI INTERVIEW COPILOT

Input:

- Candidate answer
- Interview question
- Conversation history

Output:

- Technical assessment
- Missing concepts
- Suggested follow-up
- Communication observations
- Summary

The AI should assist rather than independently make irreversible hiring decisions.

25. CODING EXECUTION

Candidate code must execute in isolated environments.

Architecture:

```

Candidate
|
Code Submission
|
API
|
Job Queue
|
Sandbox Worker

```

```
|  
Compiler/Runtime  
|  
Tests  
|  
Result
```

Never execute arbitrary candidate code inside the primary FastAPI process.

Possible isolation:

- Containers
- MicroVMs
- Dedicated sandbox workers

Security testing is mandatory.

26. API STRUCTURE

Suggested:

```
/api/v1/auth  
/api/v1/users  
/api/v1/organizations  
/api/v1/exams  
/api/v1/questions  
/api/v1/candidates  
/api/v1/sessions  
/api/v1/proctoring  
/api/v1/evidence  
/api/v1/risk  
/api/v1/interviews  
/api/v1/evaluations  
/api/v1/reports
```

Version APIs from the beginning.

27. AUTHORIZATION

Use:

RBAC + organization-level authorization.

Every protected request must verify:

1. Identity
2. Role
3. Organization
4. Resource ownership/access

Never rely on frontend authorization.

28. API SECURITY

Implement:

- HTTPS
- JWT/session security
- Input validation
- Rate limiting
- Secure headers
- CORS policy
- Request size limits
- File validation
- Audit logging
- Secret management

Sensitive endpoints require stricter controls.

29. SESSION SECURITY

Exam sessions must have:

- Unique session ID
- Candidate ID
- Exam ID
- Start timestamp
- End timestamp
- Server-side state
- Session token
- Integrity status

Important exam events should be audit logged.

30. AUDIT LOGGING

Audit:

- Login
- Logout
- Exam creation
- Exam modification
- Candidate assignment
- Exam start
- Exam submission
- Proctoring events
- Evidence access
- Result modification
- Admin actions

Audit logs should be append-oriented and protected from unauthorized modification.

31. FRONTEND ARCHITECTURE

Suggested structure:

```
apps/  
  web/  
  desktop/  
  
packages/  
  ui/  
  types/  
  config/
```

Use reusable components.

Use strict TypeScript.

Avoid duplicate types between frontend and backend where possible.

32. BACKEND STRUCTURE

Suggested:

```
backend/  
  app/  
    api/  
    core/  
    models/  
    schemas/  
    services/  
    repositories/  
    workers/  
    proctoring/  
    risk/  
    interview/  
    evaluation/  
    tests/
```

Keep business logic outside route handlers.

33. PROCTORING SERVICE STRUCTURE

```
proctoring/  
  camera/  
  audio/  
  screen/  
  identity/  
  detection/  
  events/  
  evidence/
```

Each detector should expose a clear interface.

Example:

```
ObjectDetector  
FaceDetector  
GazeDetector  
AudioDetector  
ScreenMonitor  
IdentityVerifier
```

34. MODEL INTERFACES

AI models should be replaceable.

For example:

```
ObjectDetector
  |
YOLODetector

FaceDetector
  |
MediaPipeFaceDetector

RiskModel
  |
RuleBasedRiskEngine
```

Do not tightly couple the entire platform to one model.

35. ASYNCHRONOUS PROCESSING

Heavy tasks should use workers.

Examples:

- Video processing
- Evidence generation
- Report generation
- AI evaluation
- Code execution
- Large media processing

API should remain responsive.

36. MESSAGE / JOB INFRASTRUCTURE

Initial:

Redis-based queue where appropriate.

Future:

Kafka / RabbitMQ / Redis Streams.

Do not introduce Kafka simply because the platform is intended to scale.

Introduce it when throughput and operational requirements justify it.

37. DEPLOYMENT

Development

Docker Compose:

```
Frontend
Backend
PostgreSQL
Redis
Worker
```

Production

Potential:

```
Cloud Load Balancer
      |
API Cluster
      |
Redis
      |
PostgreSQL
      |
Worker Cluster
      |
GPU Workers
      |
Object Storage
```

AWS is the initial target cloud.

38. CI/CD

GitHub Actions should eventually run:

- Lint
- Type checking
- Unit tests
- Integration tests
- Build
- Security scanning
- Docker image build

Deployment should occur only after required checks pass.

39. TESTING STRATEGY

Backend

- Unit tests
- Integration tests
- API tests

Frontend

- Component tests
- E2E tests

Desktop

- Installation tests
- Permission tests
- Camera tests
- Microphone tests
- Screen tests
- Secure-mode tests

AI

Measure:

- Precision
- Recall
- F1
- False-positive rate

- False-negative rate
- Inference latency

Load

Test:

- Concurrent candidates
- Event throughput
- WebSocket connections
- Database load
- GPU workers
- Evidence upload

40. AI QUALITY REQUIREMENTS

Every production detection model must be evaluated.

Metrics should include:

```
Precision
Recall
F1 Score
False Positive Rate
False Negative Rate
Inference Latency
```

Testing must include different:

- Lighting conditions
- Camera qualities
- Backgrounds
- Noise levels
- Device types
- Candidate positions

41. SECURITY TESTING

Security testing should include:

- Authentication testing

- Authorization testing
 - API penetration testing
 - Desktop security testing
 - Session hijacking tests
 - Tampering tests
 - File upload tests
 - Dependency scanning
 - Container scanning
 - Code execution sandbox tests
-

42. PRIVACY ARCHITECTURE

Sensitive data must have:

- Access controls
- Encryption
- Retention policy
- Audit logging
- Controlled deletion

Do not retain more data than required.

Evidence retention should be configurable.

43. OBSERVABILITY

Use:

- Structured logging
- Metrics
- Distributed tracing
- Error monitoring

Potential stack:

Prometheus

+

Grafana

+

44. PERFORMANCE TARGETS

Initial targets:

API:

<300 ms for normal lightweight requests where practical.

Realtime events:

Target sub-second alert propagation.

Exam UI:

Responsive interaction.

AI:

Model latency must be measured independently.

These are engineering targets, not guarantees.

45. SCALABILITY STRATEGY

Scale independently:

API servers

↓

Realtime servers

↓

Proctoring workers

↓

GPU workers

↓

Database

↓

Object storage

The system should avoid processing all candidates through one server.

46. FAILURE HANDLING

The system must handle:

- Camera disconnect
- Microphone disconnect
- Internet interruption
- WebRTC failure
- Backend outage
- Redis outage
- Worker failure
- Model failure

Candidate should receive clear status.

Critical exam state should not be lost.

47. OFFLINE / NETWORK INTERRUPTION

Short network interruptions should not immediately destroy an exam.

The desktop application should maintain appropriate local session state and synchronize when connectivity returns, subject to exam security policy.

Server remains authoritative for final exam state.

48. SECURITY VS USABILITY

AssessX must balance:

Security

and

Candidate experience.

Do not create unnecessary restrictions that produce large numbers of false positives or prevent legitimate candidates from completing exams.

49. ARCHITECTURAL PRINCIPLE

Start as a modular monolith where practical.

Do NOT begin with dozens of microservices.

Initial architecture:

```
FastAPI
+
PostgreSQL
+
Redis
+
Worker
+
Desktop Client
```

As scale increases, extract:

- Proctoring workers
- GPU inference
- Interview service
- Code execution
- Media processing

into independently scalable services.

50. FIRST IMPLEMENTATION PHASE

Build only:

```
Public Website
|
Download
|
Windows Desktop App
|
Login
|
Role Selection
|
Student Dashboard
Admin Dashboard
```

Backend:

```
FastAPI
PostgreSQL
Redis
```

No complex AI yet.

51. SECOND IMPLEMENTATION PHASE

Build:

```
Exam Creation
|
Question Bank
|
Candidate Assignment
|
Exam Session
|
Timer
|
Submission
|
Results
```

52. THIRD IMPLEMENTATION PHASE

Build:

System Check
Camera
Microphone
Screen
Identity
Secure Exam Mode

53. FOURTH IMPLEMENTATION PHASE

Build:

YOLO
MediaPipe
Audio Detection
Screen Events
Event Engine
Risk Engine
Evidence
Admin Monitoring

54. FIFTH IMPLEMENTATION PHASE

Build:

Coding Assessments
Collusion Detection
Advanced Analytics

55. SIXTH IMPLEMENTATION PHASE

Build:

WebRTC
Live Interview
Screen Sharing
Interview Security
Interview Reports

56. SEVENTH IMPLEMENTATION PHASE

Build:

AI Interviewer
Adaptive Questions
Interview Copilot
AI Evaluation
Candidate Scorecard

57. EIGHTH IMPLEMENTATION PHASE

Build:

Multi-tenancy
AWS scaling
GPU cluster
Advanced observability
Enterprise security

58. ENGINEERING RULES FOR CLAUDE

Claude must:

1. Read CLAUDE.md before development.
2. Read PRD before implementing product features.
3. Read TRD before changing architecture.
4. Inspect existing code before editing.
5. Avoid unnecessary rewrites.
6. Implement incrementally.

7. Write tests with features.
 8. Run tests after changes.
 9. Update documentation.
 10. Never hardcode secrets.
 11. Never execute untrusted code directly.
 12. Never claim unsupported security capabilities.
 13. Keep AI models replaceable.
 14. Keep security decisions explainable.
 15. Preserve backwards compatibility where practical.
-

59. DEFINITION OF DONE

A feature is NOT complete merely because code exists.

A feature is complete when:

- Implementation exists
 - API works
 - UI works where applicable
 - Tests exist
 - Tests pass
 - Error handling exists
 - Security considerations are addressed
 - Documentation is updated
 - Existing functionality remains working
-

60. FINAL ARCHITECTURAL PRINCIPLE

AssessX should evolve from:

Simple Exam Application

into:

Secure Assessment Platform

```
graph TD
    A[Secure Assessment Platform] --- B[Exam Engine]
    A --- C[Desktop Security]
    A --- D[AI Proctoring]
```

--- Exam Engine

--- Desktop Security

--- AI Proctoring

```
|  
+-- Risk Engine  
|  
+-- Evidence System  
|  
+-- Coding Assessment  
|  
+-- Live Interviews  
|  
+-- AI Interviews  
|  
+-- Enterprise Infrastructure
```

The architecture must remain modular, explainable, secure, privacy-conscious and scalable.

END OF TRD